

PAPER

GPU-accelerated dislocation dynamics using subcycling time-integration

To cite this article: N Bertin *et al* 2019 *Modelling Simul. Mater. Sci. Eng.* **27** 075014

View the [article online](#) for updates and enhancements.

You may also like

- [Data-driven exploration and continuum modeling of dislocation networks](#)
Markus Sudmanns, Jakob Bach, Daniel Weygand et al.
- [AsterX: a new open-source GPU-accelerated GRMHD code for dynamical spacetimes](#)
Jay V Kalinani, Liwei Ji, Lorenzo Ennoggi et al.
- [Efficient time integration in dislocation dynamics](#)
Ryan B Sills and Wei Cai

GPU-accelerated dislocation dynamics using subcycling time-integration

N Bertin^{1,2} , S Aubry², A Arsenlis² and W Cai¹

¹ Department of Mechanical Engineering, Stanford University, Stanford, CA, United States of America

² Lawrence Livermore National Laboratory, Livermore, CA, United States of America

E-mail: bertin1@llnl.gov

Received 10 May 2019, revised 5 July 2019

Accepted for publication 9 August 2019

Published 28 August 2019



CrossMark

Abstract

Discrete dislocation dynamics (DDD) simulations are being increasingly employed to investigate metal plasticity at the mesoscale. However, in spite of its ability to access beyond the length and time limits of atomistic methods, the DDD model is still limited by its high computational cost, with ranges of achievable strains too low and strain rates too high by several orders of magnitude compared with typical experimental conditions. By combining the efficiency of the recently developed subcycling time-integrator with the highly parallel architecture of graphics processing unit (GPU) devices, we developed a DDD model that provides significant acceleration compared to existing implementations. Our GPU-accelerated implementation enables large-scale DDD simulations that can reach relevant levels of strain using a moderate amount of computational resources.

Keywords: dislocation dynamics, GPU, subcycling

(Some figures may appear in colour only in the online journal)

1. Introduction

Over the last two decades, discrete dislocation dynamics (DDD) simulations, which simulate the motion and interactions of large ensembles of dislocations, have proven to be a valuable tool for the study of plasticity in metals with microscopic detail. In DDD simulations, the dislocation microstructure is discretized into a set of segments inter-connected through dislocation nodes [1]. The structure is typically evolved in time in two main stages. At every time step, the motion of dislocation nodes is first time-integrated, and topological operations are then performed on segments that have intersected, e.g. to handle junctions formation. The first stage is by far the most time consuming part of the simulation and will be the main focus

of this paper. Specifically, let $\mathbf{r}$ be the high-dimensional vector representing the position of all nodes of the DDD model. The nodal positions $\mathbf{r}$ are updated through the equation of motion (EOM)

$$\frac{d\mathbf{r}}{dt} = \mathbf{g}(\mathbf{r}) = \mathcal{M}(\mathbf{F}), \quad (1)$$

where $\mathbf{g}(\mathbf{r})$ is the velocity operator, that describes nodal velocities in response to the force $\mathbf{F}$ exerted on nodes via the mobility function $\mathcal{M}$. Nodal forces $\mathbf{F}$ are calculated by integrating the Peach–Koehler force $\mathbf{f}^{\text{PK}}$, which is proportional to the total stress $\boldsymbol{\sigma}^{\text{tot}}$ acting along dislocation line

$$\mathbf{f}^{\text{PK}} = (\boldsymbol{\sigma}^{\text{tot}}(\mathbf{r}) \cdot \mathbf{b}) \times \mathbf{t}, \quad (2)$$

where $\mathbf{b}$ and $\mathbf{t}$ are the Burgers vector and the unit tangent of the dislocation line, respectively. Since dislocations produce long-range stresses that decay as $1/R$, $\boldsymbol{\sigma}^{\text{tot}}$ in equation (2) must include the contribution of all segments present in the simulation volume (and of their periodic images when periodic boundary conditions are used), and is therefore a function of the nodal positions $\mathbf{r}$. Consequently, the calculation of nodal forces depends on the interactions between all pairs of segments. As a result, a brute-force nodal force calculation scales with $\mathcal{O}(N_{\text{seg}}^2)$, where N_{seg} is the number of dislocation segments, making it by far the most computationally intensive stage of the DDD simulation. In addition, this computational burden is further increased by the significant multiplication of dislocations in response to the deformation. Together, these behaviors severely limit the system sizes and the amounts of strain that can be simulated by DDD.

In recent years, substantial efforts have been undertaken to alleviate the computational cost of DDD simulations. These include the development of analytical expressions for interaction forces [2, 3], long-range approximation schemes that take advantage of the $1/R$ asymptotic behavior of dislocations stress (e.g. the Box Method [4] and the fast multipole method (FMM) [3, 5–9]), and full-field eigenstrain-based approaches using the finite element method (FEM) [10, 11] and fast Fourier transforms [12–14]. Concurrently, a great deal of attention has been directed to the development of massively parallel algorithms [3, 6, 15–17], e.g. the ParaDiS program [18], and a few implementations have been proposed to accelerate the calculation of nodal forces [19] and long-range stresses [13, 14] using graphics processing unit (GPU) devices.

Despite these numerous improvements, it was recently pointed out that the amount of strain that can be reached is primarily limited by the time step sizes that are required to integrate the EOM in equation (1) [20, 21]. The integration procedure in DDD is difficult because the system is intrinsically stiff and highly nonlinear due to the nature of close-range dislocation interactions. In addition, the time steps used in conventional time-integrators (e.g. Heun and Runge–Kutta–Fehlberg (RKF) schemes) are global, i.e. must apply to every node simultaneously. As a result, the global time step size is typically limited by the ‘most unstable’ dislocation node (i.e. the one that requires the smallest time step size), and the integrators are required to take increasingly small time step sizes as the dislocation network is being refined with strain.

To circumvent this issue, subcycling time-integrators were recently introduced [20, 21]. The central idea of subcycling integrators lies on isolating fast, ill-behaving modes from the rest of the system and integrate them separately. By separating interaction forces in several groups based on interaction distances, the force-based subcycling approach [21] was found to be particularly well suited for large-scale strain-hardening simulations, allowing for gains in computation time up to two orders of magnitude compared to traditional integrators. These

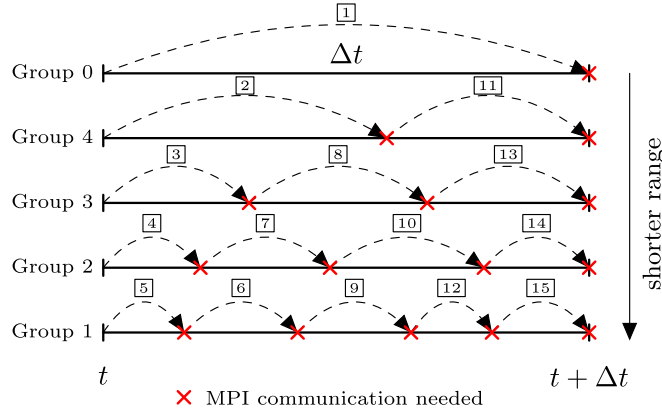


Figure 1. Schematic of the subcycling integration scheme. Segment pair interaction forces are separated into five groups based on the interaction distances. Long-range forces (Group 0) are integrated first and set the global time-step size Δt . Shorter-range interaction groups are then subcycled in a sequential fashion. The numbers in square boxes indicate the order in which subcycles are performed. In parallel calculations using the message passing interface (MPI), communications (indicated with crosses) are needed to update nodal positions between each subcycle.

gains primarily result from the fact that only a small fraction of the total interaction forces need to be subcycled, while the vast majority of interactions are well-behaving and can be integrated with large time step sizes.

However, the dramatic improvement allowed by the force-based subcycling scheme has been observed to come at the cost of a loss of scalability in parallel computations [21] with existing communication schemes based on the message passing interface (MPI) [3]. As illustrated in figure 1, this is because the cost of communications, required between each subcycle, increases relative to the cost of force calculations as the latter is dramatically reduced when integrating short-range interaction groups.

In this paper, we present a GPU-accelerated implementation of DDD simulations based on the subcycling integrator implemented in ParaDiS [3]. The use of GPU computation is particularly well-suited for the subcycling integration scheme because (i) it allows to virtually eliminate all communication costs, which are the main bottleneck to the scalability of the approach, and (ii) the number of degrees of freedom of the system remains constant during the time-integration, thereby limiting the amount of memory transfer between the host CPU and the GPU device to a minimum. Our implementation allows to speed up the DDD cycle by an additional factor of 30–40 using a single GPU device. Thus, by combining the efficiency of the subcycling integrator with the highly parallel throughput of GPU computations, our approach enables the simulations of large-scale dislocation microstructures using a single GPU. This gain in efficiency for instance allows multiple simulations to be performed concurrently (if the user has access to multiple GPUs) in order to generate statistically significant data.

2. Method and algorithms

The different stages that are performed at each global time step of a DDD simulation are summarized in figure 2. In our implementation, the focus is first placed on accelerating the

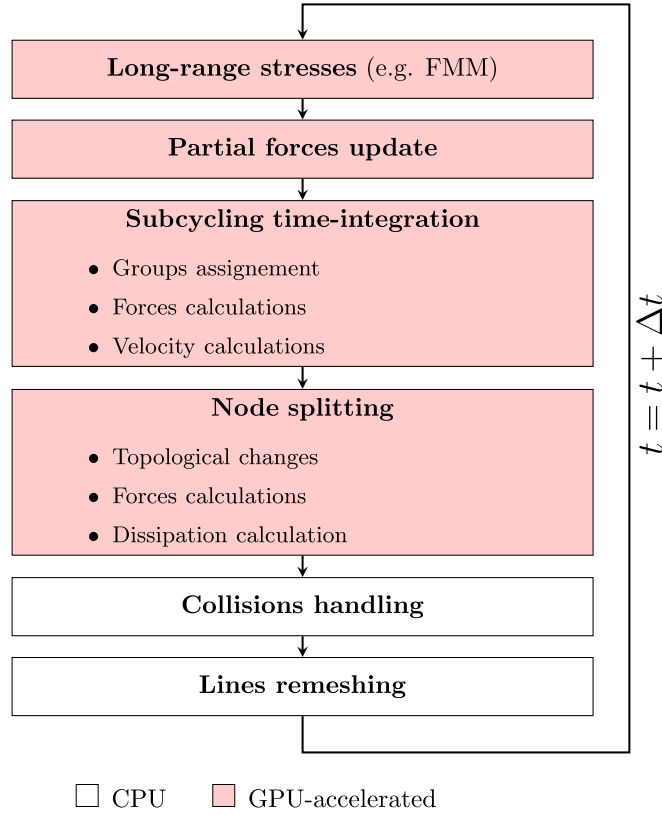


Figure 2. Main stages of the DDD cycle with subcycling integration. Stages in shaded boxes are accelerated using GPU in the current implementation. The remaining stages (white) are performed on the host CPU.

most computationally intensive stage, i.e. the time-integration procedure. This stage is central to the DDD approach and involves a large number of forces and velocities calculations for all nodes. After a GPU implementation of the subcycling integrator is developed (sections 2.1–2.5), the different kernels are used to accelerate other costly stages that involve force calculations, namely the partial forces update and node splitting operations (section 2.6). Finally, we develop a GPU-acceleration of the FMM (section 2.7) and an efficient retroactive collision algorithm (section 2.8) on CPU.

2.1. Force-based subcycling time-integrator

In the force-based subcycling approach [21], five groups are used to separate segments pair interaction forces based on interaction distances. When a linear mobility law is used, the velocity operator in equation (1) can be split into multiple terms

$$g(\mathbf{r}) = \sum_{i=0}^4 g_i(\mathbf{r}), \quad (3)$$

so that each group $i = \{0, \dots, 4\}$ can be time-integrated sequentially with independent time step sizes using an error-control scheme [3]. Specifically, the time-integration procedure is illustrated in figure 1 and proceeds as follows. First, all nodal positions are integrated by only

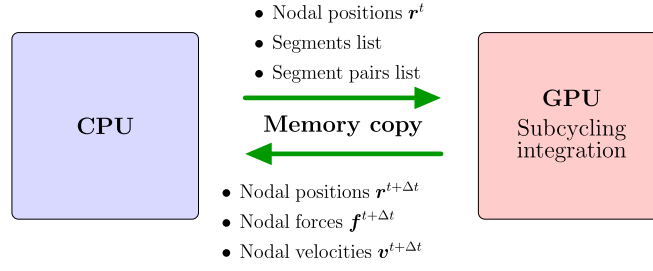


Figure 3. GPU-accelerated implementation of the force-based subcycling integrator. Nodal positions $\mathbf{r}^t$ at time t and segments and segment pairs lists are first transferred to the GPU device memory. The entire subcycling procedure is then performed on the GPU device. After integration, all nodal information at time $t + \Delta t$ are transferred back to the host CPU memory, where Δt is the global time step.

considering the interaction forces contained in the long-range, well-behaving interaction group 0, and the global time-step size Δt is determined. Nodal positions are then advanced by considering, in turn, the (shorter range) interaction forces contained in the remaining groups 1 to 4. Since the time-step size associated with short-range groups is likely to be smaller than that of the long-range group, the integration of these groups is performed iteratively until every group catches up in time with group 0.

In order to maximize the efficiency of our GPU-accelerated approach, the entire subcycling procedure is performed on the GPU device. This design is facilitated by the fact that no topological operation is performed during time-integration, hence the number of degrees of freedom (nodal positions) and the segments connectivity remain constant during the whole subcycling procedure. Thus, memory transfers between the host CPU and the GPU device, which often act as a bottleneck in GPU-accelerated applications, are reduced to a minimum. In fact, any strategy requiring memory transfers between subcycles would be doomed to fail for the same reasons that the current subcycling implementation does not scale well on multiple CPUs using the message passing interface (MPI).

Our GPU implementation of subcycling is illustrated in figure 3. Nodal positions $\mathbf{r}^t$ at time t and the lists of segments and segment pairs are transferred to the GPU memory once at the beginning of the integration cycle. The whole subcycling integration, whereby nodal positions are sequentially advanced in time by switching between the different interaction groups, is then performed without requiring any memory transfer between the CPU and the GPU. For each subcycle, time-integration is performed using a fifth-order RKF integrator fully implemented on the GPU. Once time-integration is completed for all groups, all nodal information (positions, forces, velocities) at time $t + \Delta t$ are transferred back to the CPU memory for the topological operations (e.g. collisions, remeshing) to be handled.

In the following, the kernel functions implemented in CUDA and associated with the three main components of the subcycling integrator, namely groups assignment, forces calculation, and velocities calculation, are presented in more details.

2.2. Groups assignment

In the force-based subcycling scheme proposed in [21], the grouping of segment pairs interactions is based on the interaction distance. For each pair of segments, the interaction distance is defined as the minimum distance between both segments in the pair. As a result, groups are likely to have very different sizes, with typically millions of pairs in group 0 and

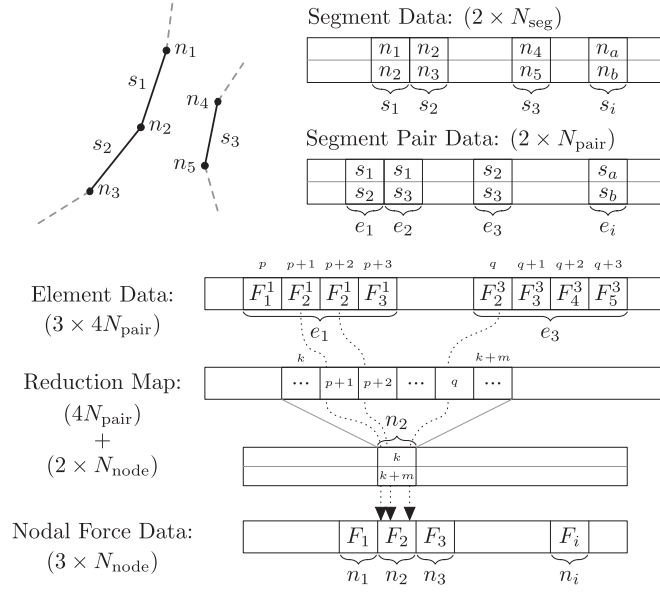


Figure 4. Data structure for forces calculation on the GPU device. The segments list is represented as an array of N_{seg} 2-tuples specifying the indices of segments end nodes. For each group, the list of segment pairs is represented as of an array of 2-tuples specifying the indices of both segments in the pair. The total force F_i on each node n_i is assembled from the set of element interaction forces $\{F_i^e\}$ stored in the global element data array using the reduction map arrays.

only few thousands in short-range groups. As such, constructing the list of segment pairs on the fly from the segments list on the GPU would be highly inefficient and would make load balancing impractical. In addition, our implementation uses the FMM to approximate far-field interactions, such that certain segment pairs need to be excluded from the short-range interaction list in order to avoid double-counting of forces.

In our implementation, groups are entirely determined before starting the subcycling procedure, and a list of segment pairs is associated with each group. To avoid complexities associated with implementing the FMM cells structure on the GPU, the list of all segment pairs is first populated on the CPU. This procedure is relatively fast as no arithmetical operation is involved. The list of segment pairs is represented as a vector of 2-tuples specifying the indices of both segments in the pair (see figure 4). Initially, all interactions are assigned to group 0. The list is then transferred to the GPU, where pair distances are evaluated efficiently. In doing so, each interaction is assigned to a group based on its distance and the grouping radius r_{gi} associated with each group i (see [21]). While most of the interactions typically remain in group 0, a fraction of these that have been flagged to groups 1–4 are moved to the segment pairs lists associated with these groups.

2.3. RKF kernel

For each subcycle, the time-integration is performed using a fifth-order RKF method. The RKF scheme requires six intermediate force and velocity evaluations to gradually build up the solution for the nodal positions during each subcycle (see [20]). The nodal positions are incrementally updated using the force and velocity kernels presented in the following sections

sections 2.4 and 2.5. In order to evaluate the error and gradually build up the solution during each subcycle, the intermediate nodal velocities are stored in an array of size $3 \times 6N_{\text{node}}$. An additional array of size N_{node} is used to implement the ‘forward progress check’ approach detailed in [21] to avoid spurious oscillations of nodes around their equilibrium position. This array is also used to flag troublesome nodes that may fail to converge during integration of group 0. All interactions associated with these troublesome nodes are transferred to group 4 in order for the global time step size Δt to remain as large as possible (see [21] for more details).

2.4. Forces kernel

The forces calculation kernel operates in a way analogous to the assembly procedure in a traditional FEM program. It takes in a list of N_{pair} segments pairs and a reduction map. Each interaction pair is considered as a 4-nodes element. Nodal forces for each element are first evaluated using the non-singular interaction force formulation [2] by assigning one thread per interaction, and the resulting values are stored in a global element array of size $3 \times 4N_{\text{pair}}$ to avoid race conditions. The assembly of the total forces on each node from the element forces is then performed by assigning one thread per node using the reduction map. As illustrated in figure 4, the reduction map is composed of two arrays. The first array has size $4N_{\text{pair}}$ and contains the positions of all forces in the global element array that need to be fetched to calculate the total force on each node. Since all nodes might be involved in a different number of interactions, we use an auxiliary array of N_{node} 2-tuples that contains the indices range associated with each node in the position array. To increase efficiency, the reduction map is built on the GPU after segment pairs lists have been transferred from the CPU.

In principle, more optimized assembly algorithms (e.g. using shared memory and coalesced reading, see [22]) could be used. However, the cost of the kernel is largely dominated by the interaction force calculation, which involves more than 1500 arithmetic operations [3] with high register pressure. As such, optimizing global memory access would only have a low impact on the overall performance in this case.

As discussed in [21], the grouping radius r_{g1} of group 1 is set to zero so that self-interactions are assigned to it, and group 1 also contains the contributions from the core energy, the applied stress, and the far-field interactions computed with the FMM. As a result, group 1 has a dedicated kernel in our implementation. To avoid developing a GPU implementation of the FMM stress evaluation while eliminating memory transfers, the FMM stress contributions are pre-computed on the CPU at several Gauss points along each dislocation segment before entering the subcycling loop, i.e. far-field stresses are assumed to be constant during integration. Since the FMM contribution is a slowly varying field, this approximation is not expected to induce any noticeable effect. During integration, the corresponding FMM Peach–Koehler forces are re-evaluated from the FMM stresses during each subcycle to properly account for the effects of segments rotation.

2.5. Velocity kernel

Once forces $\mathbf{F}_i$ are calculated on each node i , nodal velocities $\mathbf{V}_i$ are obtained using the mobility function $\mathcal{M}$ introduced in equation (1). For an overdamped EOM, the nodal force–velocity relation is described by the mobility law

$$\mathbf{V}_i = \mathbf{M}(\mathbf{F}_i, \{g_i^c\}), \quad (4)$$

where $\mathbf{M}$ is the nodal mobility operator, and $\{g_i^c\}$ denotes the set of glide constraints associated with node i .

2.5.1. Linear scalar mobility law. As an example, let us consider a linear scalar mobility law, e.g. such as these typically used in FCC crystals [1]. In this mobility law, velocities are assumed to be orthogonal to the dislocation lines and glide constraints are explicitly enforced by projecting nodal velocities onto their glide planes. Thus, for the sake of efficiency, glide planes constraints are pre-computed such as to recast the mobility function in equation (4) as a fast matrix-vector multiplication operator on the GPU. Specifically, the nodal force–velocity relation can be written as

$$\mathbf{V}_i = \mathbf{M}_i \mathbf{F}_i, \quad (5)$$

where $\mathbf{M}_i$ is a 3×3 matrix that can be expressed as the product of two terms

$$\mathbf{M}_i = \left(\prod_c^{n_c} \mathbf{P}_c \right) \mathbf{B}_i. \quad (6)$$

The first term is the product of the projectors $\mathbf{P}_c$ associated with all n_c nodal glide constraints. For glide constraint with normal plane $\mathbf{n}^c$, the projector $\mathbf{P}_c$ is given by

$$\mathbf{P}_c = \mathcal{I}_2 - \frac{\mathbf{n}_\perp^c \otimes \mathbf{n}_\perp^c}{\|\mathbf{n}_\perp^c\|^2}, \quad (7)$$

where $\mathbf{n}_\perp^c$ indicates that the set of glide plane vectors $\{\mathbf{n}^c\}$ have been orthogonalized among themselves, and $\mathcal{I}_2$ is the 3×3 identity matrix. Since no topological operation is performed during time-integration, this first term can be pre-computed for each node before entering the subcycling loop.

Assuming smooth dislocation lines, the second term $\mathbf{B}_i$ in equation (6) denoting the nodal drag matrix can be expressed as [1]

$$\mathbf{B}_i = \left(\frac{1}{2} \sum_j L_{ij} \mathcal{B} \right)^{-1} \mathcal{I}_2, \quad (8)$$

where the sum is taken over all nodes j connected to node i , L_{ij} is the length of segment ij , and $\mathcal{B}$ is a scalar drag coefficient. Since this term depends on segment lengths, i.e. nodal positions, it cannot be pre-computed. Here, the drag coefficient $\mathcal{B}$ is solely a function of the dislocation orientation (character angle) and does not depend on the force. Similarly to the nodal forces assembly kernel, the drag matrix in equation (8) is first calculated for each segment ij and stored in a global drag array of size N_{seg} , and then assembled at nodes using a segment reduction map.

2.5.2. Nonlinear scalar mobility laws. The validity of the splitting of the velocity operator in equation (3) implicitly entails the use of a linear mobility law. Clearly, this can be seen by noting that while the force–velocity relation is necessarily a function of the total force acting on dislocations, only a subset of forces are considered during time-integration of individual groups in the subcycling approach.

Here, we present an extension of the original subcycling integrator to include nonlinear scalar mobility laws based on a tangent velocity approximation scheme. As illustrated in figure 5, the velocity at each force value can be approximated by considering a linear mobility law whose effective drag coefficient $\mathcal{B}^*$ is given by the slope of the scalar force–velocity relation at this point. With this scheme, the force f^* at which the tangent mobility intersects the zero-velocity axis can be regarded as the magnitude of a back-force acting against dislocation motion. Thus, for each node, the effective drag coefficient $\mathcal{B}^*$ and the back-force

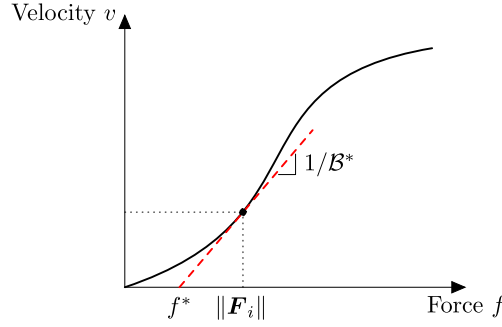


Figure 5. Schematic of the tangent mobility approximation developed to enable the use of nonlinear scalar mobility laws with subcycling integration. For each nodal force value $\|\mathbf{F}_i\|$, the mobility law is linearized using a tangent approximation with effective drag $\mathcal{B}^*$. The intersection between the tangent and the zero-velocity axis defines an effective back-force of magnitude f^* acting on the node.

$\mathbf{F}_i^*$ defined as

$$\mathbf{F}_i^* = -\frac{\mathbf{F}_i^t}{\|\mathbf{F}_i^t\|} f^* \quad (9)$$

are determined from the total nodal force $\mathbf{F}_i^t$ at time t before entering the subcycling integration procedure. Following equation (9), the effective nodal force is given by $\mathbf{F}_i^t + \mathbf{F}_i^*$. During integration, the effective drag coefficient $\mathcal{B}^*$ is used to evaluate the drag matrix in equation (8) for all groups, while nodal back-forces are treated as any other force contributions and are added to the nodal forces. Specifically, we found that including the back-force contribution in group 1 (i.e. along with the applied stress and FMM contributions) leads to the best accuracy.

Note that this scheme does not require any supplementary force calculation as the total force $\mathbf{F}_i^t$ on each node i is always known before integration in ParaDiS [3] (see section 2.6). Regarding the GPU implementation, this extension only requires two supplementary arrays of size N_{node} and $3 \times N_{\text{node}}$ containing the effective drag value and the pre-computed back-force contribution associated with each node, respectively.

2.6. Node splitting and partial forces update

While the force and velocity kernels presented in sections 2.4 and 2.5 are primarily used in the RKF integrator to advance nodal positions during subcycling, they can also be conveniently used to accelerate other stages of the DDD cycle. Specifically, the node splitting and partial forces update stages, which both involve numerous pair interaction calculations, are also accelerated using the GPU in our implementation.

As discussed in [3], the node splitting procedure is used to determine whether multi-arm nodes that have formed in collision events should be dissociated. In this process, all possible dissociation configurations are tested and the one that maximizes the dissipation is selected. Thus, the node splitting procedure requires forces on the dissociated nodes to be fully recalculated for each trial configuration in order for the dissipation to be evaluated. Here, we use the GPU to accelerate the calculation of segment pairs interaction forces associated with the dissociated nodes, while the other contributions (self-force, applied and FMM stresses) remain calculated on the CPU. Since the segment connectivity is different for every trial

configuration, we use a sequential implementation; for each configuration, the list of pair interactions associated with the dissociated nodes is first populated on the CPU. The segment pairs list is then transferred to the GPU where the nodal forces and velocities are evaluated and assembled using the kernels presented in sections 2.4 and 2.5. The nodal values are then copied back to the CPU memory where the dissipation is calculated before handling the next trial configuration.

The partial forces update stage aims at recomputing forces on nodes that have undergone topological changes to ensure that all nodal forces remains accurate before entering the integration procedure. Here again, the GPU is used to accelerate the calculation of segment pairs interactions associated with all nodes that have undergone topological changes. The list of all segment pairs is first populated on the CPU, and nodal forces are then evaluated on the GPU using the kernel presented in section 2.4.

2.7. Fast multipole method

Our implementation uses the FMM approach to compute far-field dislocations stress fields. One of the most computationally intensive stage of the FMM is the computation of the multipole moments associated with each cell, that form the coefficients of the Taylor expansion of the dislocation segments stress. At each level of the FMM hierarchy, the calculation of the Taylor coefficients requires $189N_c p^4$ operations, where p is the order of the expansion, N_c is the number of cells in the hierarchy level, and 189 is the number of neighboring cells usually considered in the calculation. As such, this stage alone might take a significant fraction of the DDD cycle time, especially when a fine FMM grid is used and when the computation is performed using a single host CPU. In addition, we note that, within each level, the calculation of the Taylor coefficients for each pair of cell-neighbor cell are independent and can therefore be computed in parallel. In our implementation, we therefore accelerate this stage by computing, for each level, the $189N_c$ Taylor coefficients contributions (see [3]) in parallel on the GPU, and use CUDA atomic operations to aggregate the contributions for each cell directly during the execution of the kernel.

2.8. Retroactive collision algorithm

The goal of collision detection algorithms is to determine if two dislocation segments collide with each other within a given time interval. In the context of the subcycling approach where the global time step Δt may be large, it is necessary to develop a robust and efficient collision algorithm to ensure that no collision is missed.

Here we present a modified and more efficient version of the original retroactive collision algorithm introduced in [21]. The collision occurs if the two dislocation segments are currently within a pre-specified distance, or were within that distance during the course of the last time increment (i.e. by looking retroactively). If the collision occurs, then the shortest distance between the segments at collision time will be computed.

The algorithm consists in finding, during time interval $[t, t + \Delta t]$, the minimum distance between two moving positions $\mathbf{P}(\tau)$ and $\mathbf{Q}(\tau)$, each located on a segment defined at time $\tau = t$ as $[\mathbf{r}_1^t, \mathbf{r}_2^t]$ and $[\mathbf{r}_3^t, \mathbf{r}_4^t]$ (before time-integration), and at time $\tau = t + \Delta t$ as $[\mathbf{r}_1^{t+\Delta t}, \mathbf{r}_2^{t+\Delta t}]$ and $[\mathbf{r}_3^{t+\Delta t}, \mathbf{r}_4^{t+\Delta t}]$ (after time-integration), respectively (see figure A1(a)). These positions on the segments are

$$\begin{aligned} \mathbf{P}(\tau) &= \mathbf{r}_1(\tau)(1 - u) + \mathbf{r}_2(\tau)u \\ \mathbf{Q}(\tau) &= \mathbf{r}_3(\tau)(1 - v) + \mathbf{r}_4(\tau)v, \end{aligned} \quad (10)$$

where

$$\mathbf{r}_i(\tau) = (1 - w)\mathbf{r}_i^t + w\mathbf{r}_i^{t+\Delta t}, \quad i = \{1, \dots, 4\} \quad (11)$$

with $w = \frac{\tau-t}{\Delta t}$ parameterizing time, and $0 \leq u \leq 1$ and $0 \leq v \leq 1$ parameterizing points on the two segments, respectively. Here we have made the assumption that the dislocation nodes move with a constant velocity during the time interval $[t, t + \Delta t]$.

A spherical filter around each moving segment is first used to eliminate most pairs of segments from further consideration. Two spheres are defined such that they contain the points $(\mathbf{r}_1^t, \mathbf{r}_2^t, \mathbf{r}_1^{t+\Delta t}, \mathbf{r}_2^{t+\Delta t})$, and $(\mathbf{r}_3^t, \mathbf{r}_4^t, \mathbf{r}_3^{t+\Delta t}, \mathbf{r}_4^{t+\Delta t})$ respectively. For pairs $(\mathbf{P}(\tau), \mathbf{Q}(\tau))$ that pass the filter, i.e. for pairs for which the spheres have a non-zero intersection, a minimum distance computation is conducted. A set of special bounding cases for the pair of segments is first considered followed by a search for a general interior solution.

The general interior solution is given as solution of the minimization problem

$$\min_{0 \leq u, v, w \leq 1} F(u, v, w), \quad (12)$$

where

$$\begin{aligned} F(u, v, w) &\equiv \|\mathbf{P}(\tau) - \mathbf{Q}(\tau)\|^2 \\ &= \|\{[(1 - w)\mathbf{r}_1^t + w\mathbf{r}_1^{t+\Delta t}](1 - u) \\ &\quad + [(1 - w)\mathbf{r}_2^t + w\mathbf{r}_2^{t+\Delta t}]u\} \\ &\quad - \{[(1 - w)\mathbf{r}_3^t + w\mathbf{r}_3^{t+\Delta t}](1 - v) \\ &\quad + [(1 - w)\mathbf{r}_4^t + w\mathbf{r}_4^{t+\Delta t}]v\}\|^2. \end{aligned} \quad (13)$$

The strategy to determine if the segments are colliding during the time interval consists in finding a distance between the segments during the time interval that satisfies the preset collision distance. If at any time the distance criterion is satisfied, then the segments are subjected to a collision procedure. Determining the minimum distance requires finding any of the special or general solutions of equation (12), which yields the positions on the segments and the time of closest approach (see [appendix](#) for more details).

3. Results

An example of the performance of our GPU-accelerated implementation of DDD with sub-cycling is shown in figure 6. In this plot, the wall clock time of the original subcycling implementation running on 1 Intel Xeon X5650 CPU at 2.67 GHz is compared with the GPU-accelerated implementation executed on 1 CPU and 1 NVidia GeForce GTX Titan X GPU device using double precision kernels. In these simulations, a $(15 \mu\text{m})^3$ FCC Cu configuration with initial dislocation density $\rho_0 = 0.7 \times 10^{12} \text{ m}^{-2}$ is loaded at a strain rate of $\dot{\epsilon} = 10^3 \text{ s}^{-1}$ in the $[001]$ direction. The initial configuration is composed of about 2000 segments, and a linear mobility is used with a drag coefficient of $\mathcal{B} = 15.6 \mu\text{Pa s}$. To focus on the efficiency of the integrator, far-field interactions are computed with a coarse FMM grid of 4^3 cells, such that a large number of segment pairs interactions are calculated explicitly. When using one CPU, the shear strain value of $\gamma = 0.8\%$ is reached after about 1200 CPU hours (50 CPU days), at which stage the dislocation density reaches $\rho = 1.7 \times 10^{12} \text{ m}^{-2}$ with about 10 000 dislocation segments. This performance already corresponds to an improvement of approximately two orders of magnitude compared to a simple Heun integrator [21]. In

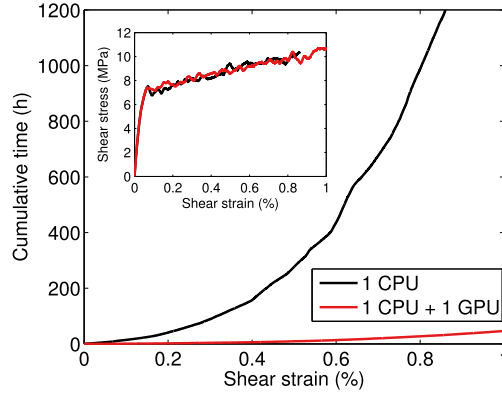


Figure 6. Wall clock time as a function of the shear strain γ for DDD simulations on a $(15\ \mu\text{m})^3$ configuration loaded at $\dot{\epsilon} = 10^3\ \text{s}^{-1}$. The GPU-accelerated approach with 1 CPU + 1 GPU (red) yields a speed-up factor of about $40\times$ compared to the original subcycling approach executed on 1 CPU (black). The inset shows that similar stress-strain responses are obtained in both cases.

Table 1. Approximate wall clock time to reach $\gamma = 1\%$ shear strain on $(15\ \mu\text{m})^3$ simulation boxes with initial dislocation density $\rho_0 = 0.7 \times 10^{12}\ \text{m}^{-2}$ and 8^3 FMM cells using 1 CPU and 1 GPU as a function of the applied strain rate $\dot{\epsilon}$.

Strain rate $\dot{\epsilon}$	Wall clock time
$10^4\ \text{s}^{-1}$	3 h
$10^3\ \text{s}^{-1}$	1 d
$10^2\ \text{s}^{-1}$	15 d

comparison, the GPU-accelerated version reaches the same level of strain in about 32 h, thereby providing an additional speed-up factor of approximately $37\times$.

In details, the effect of accelerating the different stages of the DDD cycle with the GPU can be summarized as follows. In the CPU version, the subcycling time-integration amounts to about 98% of the total computation time. Our GPU-implementation allows to accelerate the time-integration stage alone by a factor $50\times$. As such, we observe that the time-integration stage drops to approximately 50% of the total time. As a result, the relative cost of the remaining stages becomes prominent, especially for these that involve segment pair interactions such as the node splitting and partial forces update procedures. Therefore, we see that accelerating these other stages of the cycle cannot be neglected if one wants to retain the speed-up obtained during time-integration in the overall performance. In addition, we also note that the calculation of pair-wise distances involved in the group assignment procedure can lead to significant gains when executed on the GPU.

As an example, wall-clock times required to reach $\gamma = 1\%$ shear strain on $(15\ \mu\text{m})^3$ simulation boxes with 8^3 FMM cells using 1 CPU and 1 NVidia GeForce GTX Titan X GPU device are reported in table 1 for different strain rates.

As another benchmark, we report the performance of another GPU-accelerated DDD simulation in a different parameter space. Specifically, we now consider a $(1.8\ \mu\text{m})^3$ FCC Al configuration with a high initial dislocation density of $\rho_0 = 0.4 \times 10^{14}$ and a fine maximum

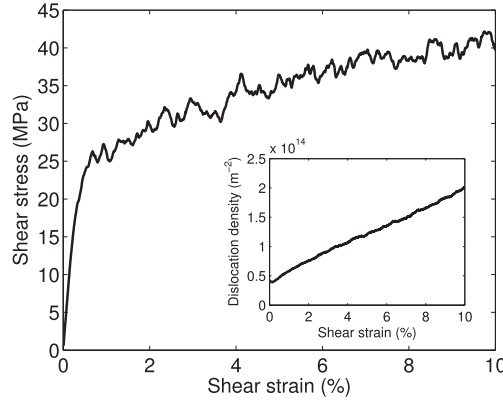


Figure 7. Shear stress–strain curve and dislocation density evolution for a $(1.8 \mu\text{m})^3$ FCC Al configuration loaded at a strain rate of $\dot{\epsilon} = 10^5 \text{ s}^{-1}$ in the $[0\ 0\ 1]$ direction up to $\gamma = 10\%$ shear strain. The simulation took 10 days to complete using our GPU-accelerated implementation.

segment discretization size of $50b$, where $b = 0.286 \text{ nm}$ is the magnitude of the Burgers vector in Al. Initially the simulation contains 20 000 dislocation segments. The configuration is then loaded at a strain rate of $\dot{\epsilon} = 10^5 \text{ s}^{-1}$ in the $[0\ 0\ 1]$ direction up to $\gamma = 10\%$ shear strain. During this process, the number of FMM cells is gradually increased from 8^3 to 32^3 . The stress–strain and density evolution for this simulation are reported in figure 7. This massive simulation took only 10 days to reach $\gamma = 10\%$ using 1 CPU and 1 GPU. At $\gamma = 10\%$, the dislocation density has reached $\rho = 2 \times 10^{14} \text{ m}^{-2}$ and there are approximately 130 000 segments in the volume.

In our proposed GPU implementation, the memory usage of the GPU device is dominated by the memory associated with segment pairs data (including the segment pair indices, segment pair forces, and reduction map arrays), with each segment pair requiring about 150 bytes of memory. For instance, about 8×10^7 segment pairs can thus be handled on a GPU device with 12 GB of memory (e.g. NVidia GeForce GTX Titan X or Tesla K40). When using a total of N_{FMM} FMM cells, the number of pair interactions N_{pair} can be estimated from the number of segments N_{seg} as

$$N_{\text{pair}} = \frac{27}{N_{\text{FMM}}} \frac{N_{\text{seg}}^2}{2}. \quad (14)$$

Thus, the number of segments that can be treated with our current GPU implementation ranges from $N_{\text{seg}} \sim 20\,000$ for $N_{\text{FMM}} = 4^3$ to $N_{\text{seg}} \sim 450\,000$ for $N_{\text{FMM}} = 32^3$ (for a GPU device with 12 GB of memory).

These results show that our GPU-accelerated implementation enables to perform large-scale DDD simulations that can reach relevant amount of strains very efficiently, while requiring limited computational resources. For instance, these efficiency gains mean that more simulations can be run to obtain statistically significant data. Our approach has been successfully used recently to generate complex strain-hardened dislocation structures and investigate their network structure [23] and their energetics [24].

4. Conclusion

We developed a GPU-accelerated implementation of DDD simulations that uses the newly developed subcycling time-integrator. By leveraging the power of massively parallel calculations offered by general-purpose GPU computing, our implementation allows to circumvent the loss of scalability associated with the current MPI parallelization of the subcycling approach, and allows to further accelerate DDD simulations by more than one order of magnitude.

Finally, we note that the method developed in this work was implemented for a single GPU device. While the development of a multi-GPU implementation could certainly lead to even larger computational gains, it would also require a careful implementation of communications between the GPUs. Additionally, we note that further gains could also be obtained by using multi-threading strategies (e.g. OpenMP) to execute the parts of the DDD cycle that remain on the CPU.

Acknowledgments

This work was supported by the US Department of Energy, Office of Basic Energy Sciences, Division of Materials Sciences and Engineering under Award No. DE-SC0010412 (NB and WC). This work was also performed in part under the auspices of the US Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344 (NB, SA, AA).

Appendix. Solution in the retroactive collision algorithm

Following the approach described in section 2.8, the strategy to determine if two segments are colliding during time interval $[t, t + \Delta t]$ consists in finding a distance between the moving positions $(\mathbf{P}(\tau), \mathbf{Q}(\tau))$ during the time interval that satisfies the preset collision distance, as depicted in figure A1. This amounts to determining whether the minimum of a three-dimensional function $F(u, v, w)$ within a unit cube, $0 \leq u, v, w \leq 1$, is within a pre-specified tolerance. The minimum of $F(u, v, w)$ may occur at the eight corners, twelve edges, six faces, or within the interior of the cube, see figure A1(b). For the sake of efficiency, the algorithm checks all of these cases in this order, and updates the minimum distance prediction with the smallest $F(u, v, w)$. As soon as the minimum distance obtained so far has become less than the pre-specified tolerance, the algorithm exits by declaring a collision has occurred, without checking the subsequent cases.

Evaluating $F(u, v, w)$ at the eight corners of the cube, i.e. $u = 0$ or 1 , $v = 0$ or 1 , $w = 0$ or 1 , amounts to computing the distance between two points, and can be evaluated easily. For example, $F(0, 0, 0) = \|\mathbf{r}_1^t - \mathbf{r}_3^t\|$.

Minimizing $F(u, v, w)$ along the 12 lines of the cube requires one-dimensional minimization procedures. For example, along the line of $u = 0$, $w = 0$, $F(u, v, w)$ reduces to

$$F(0, v, 0) = \|(\mathbf{r}_1^t - \mathbf{r}_3^t) + (\mathbf{r}_3^t - \mathbf{r}_4^t)v\|^2. \quad (\text{A.1})$$

Along the line of $u = 0$, $v = 0$, $F(u, v, w)$ reduces to

$$F(0, 0, w) = \|(\mathbf{r}_1^t - \mathbf{r}_3^t) + (\mathbf{r}_1^{t+\Delta t} - \mathbf{r}_1^t - \mathbf{r}_3^{t+\Delta t} + \mathbf{r}_3^t)w\|^2. \quad (\text{A.2})$$

It can be seen that in these cases the function has the same analytic form, and the minimum can be obtained by finding the solution to the equations $dF(0, v, 0)/dv = 0$ ($dF(0, 0, w)/dw = 0$, or $dF(u, 0, 0)/du = 0$) and check if the solution is within the domain of $(0, 1)$. If the solution is

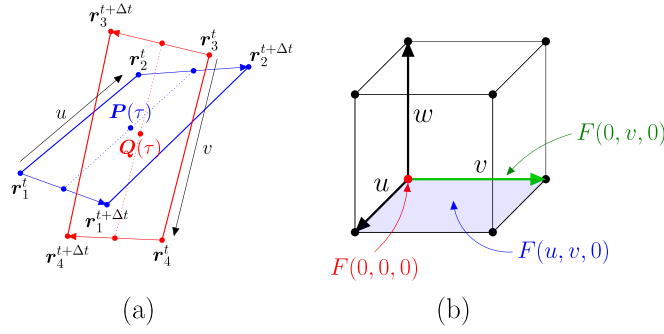


Figure A1. (a) Schematic of two dislocation segments $[r_1(\tau), r_2(\tau)]$ and $[r_3(\tau), r_4(\tau)]$ moving during time interval $\tau \in [t, t + \Delta t]$. The collision detection algorithm consists in finding if the closest approach between the two moving positions ($P(\tau)$, $Q(\tau)$) during the time interval satisfies the preset collision distance. (b) This corresponds to minimizing function $F(u, v, w)$ in equation (13) on the unit cube $0 \leq u, v, w \leq 1$. To save time, special cases corresponding to vertices, edges and faces of the cubes are tested before attempting to solve for the general interior solution.

outside the domain of $(0, 1)$, then the solution is discarded and the previous minimum distance is retained; otherwise the solution is considered viable.

Minimizing $F(u, v, w)$ on the six faces of the cube requires two-dimensional minimization procedures. For example, on the face $w = 0$, $F(u, v, w)$ reduces to the distance squared between two line segments

$$F(u, v, 0) = \|[r_1^t(1 - u) + r_2^t u] - [r_3^t(1 - v) + r_4^t v]\|^2. \quad (\text{A.3})$$

The minimum can be obtained by finding the solution to the equations $\partial F(u, v, 0)/\partial u = 0$ and $\partial F(u, v, 0)/\partial v = 0$. The two equations are linear in u and v , and the solution is viable if $u_{\min}$ and $v_{\min}$ are both within the interval $(0, 1)$. If the two equations are not linearly independent, then the search for the solution in this case is terminated.

On the other cube faces where u or v remains constant, the form of $F(u, v, w)$ becomes different. For example, on the face $u = 0$, $F(u, v, w)$ reduces to the distance between a moving point $r_1(\tau)$ and a moving segment $Q(\tau)$

$$\begin{aligned} F(0, v, w) = & \|[(1 - w)r_1^t + w r_1^{t+\Delta t}] \\ & - \{[(1 - w)r_3^t + w r_3^{t+\Delta t}](1 - v) \\ & + [(1 - w)r_4^t + w r_4^{t+\Delta t}]v\}\|^2. \end{aligned} \quad (\text{A.4})$$

The minimum is obtained by finding the solution to the equations $\partial F(0, v, w)/\partial v = 0$ and $\partial F(0, v, w)/\partial w = 0$. This is accomplished using a Newton algorithm, in which the initial guess for $v_{\min}$ and $w_{\min}$ is taken from the current minimum distance computed from all of the previous cases. The solution is considered viable if it is within the interval of $(0, 1)$. There is a potential for a degenerate solution where the minimum distance does not vary in time. If a degeneracy is detected, the attempt at finding this solution is terminated.

Finally, after all special cases have been tested, the algorithm attempts to solve for the general interior solution. Minimizing $F(u, v, w)$ at the cube interior requires solving a set of nonlinear equations: $\partial F(u, v, w)/\partial u = 0$, $\partial F(u, v, w)/\partial v = 0$, and $\partial F(u, v, w)/\partial w = 0$. A Newton algorithm is used here as well. The solution is considered viable if $u_{\min}$, $v_{\min}$ and $w_{\min}$

all fall within the interval (0, 1). Likewise, if a degeneracy is detected, the attempt at finding this solution is terminated.

ORCID iDs

N Bertin  <https://orcid.org/0000-0002-1901-8767>

References

- [1] Bulatov V and Cai W 2006 *Computer Simulations of Dislocations* vol 3 (Oxford: Oxford University Press)
- [2] Cai W, Arsenlis A, Weinberger C R and Bulatov V V 2006 A non-singular continuum theory of dislocations *J. Mech. Phys. Solids* **54** 561–87
- [3] Arsenlis A, Cai W, Tang M, Rhee M, Oppelstrup T, Hommes G, Pierce T G and Bulatov V V 2007 Enabling strain hardening simulations with dislocation dynamics *Modelling Simul. Mater. Sci. Eng.* **15** 553–95
- [4] Verdier M, Fivel M and Groma I 1998 Mesoscopic scale simulation of dislocation dynamics in fcc metals: principles and applications *Modelling Simul. Mater. Sci. Eng.* **6** 755–70
- [5] Wang H Y and Lesar R 1995 O(N) algorithm for dislocation dynamics *Phil. Mag. A* **71** 149–64
- [6] Zbib H M, Rhee M and Hirth J P 1998 On plastic deformation and the dynamics of 3d dislocations *Int. J. Mech. Sci.* **40** 113–27
- [7] LeSar R and Rickman J M 2002 Multipole expansion of dislocation interactions: application to discrete dislocations *Phys. Rev. B* **65** 144110
- [8] Zhao D, Huang J and Xiang Y 2010 A new version fast multipole method for evaluating the stress field of dislocation ensembles *Modelling Simul. Mater. Sci. Eng.* **18** 045006
- [9] Chen C, Aubry S, Oppelstrup T, Arsenlis A and Darve E 2018 Fast algorithms for evaluating the stress field of dislocation lines in anisotropic elastic media *Modelling Simul. Mater. Sci. Eng.* **26** 045007
- [10] Lemarchand C, Devincere B and Kubin L P 2001 Homogenization method for a discrete-continuum simulation of dislocation dynamics *J. Mech. Phys. Solids* **49** 1969–82
- [11] Vattre A, Devincere B, Feyel F, Gatti R, Groh S, Jamond O and Roos A 2014 Modelling crystal plasticity by 3d dislocation dynamics and the finite element method: the discrete-continuous model revisited *J. Mech. Phys. Solids* **63** 491–505
- [12] Bertin N, Upadhyay M V, Pradalier C and Capolungo L 2015 A FFT-based formulation for efficient mechanical fields computation in isotropic and anisotropic periodic discrete dislocation dynamics *Modelling Simul. Mater. Sci. Eng.* **23** 065009
- [13] Bertin N and Capolungo L 2018 A FFT-based formulation for discrete dislocation dynamics in heterogeneous media *J. Comput. Phys.* **355** 366–84
- [14] Bertin N 2019 Connecting discrete and continuum dislocation mechanics: a non-singular spectral framework *Int. J. Plast.* submitted (<https://doi.org/10.1016/j.ijplas.2018.12.006>)
- [15] Cai W, Bulatov V V, Pierce T G, Hiratani M, Rhee M, Bartelt M and Tang M 2004 Massively-parallel dislocation dynamics simulations *IUTAM Symposium on Mesoscopic Dynamics of Fracture Process and Materials Strength* (Berlin: Springer) pp 1–11
- [16] Wang Z, Ghoniem N, Swaminarayan S and LeSar R 2006 A parallel algorithm for 3d dislocation dynamics *J. Comput. Phys.* **219** 608–21
- [17] Leiter K W, Crone J C and Knap J 2013 An algorithm for massively parallel dislocation dynamics simulations of small scale plasticity *J. Comput. Sci.* **4** 401–11
- [18] ParaDiS <http://paradis.stanford.edu>
- [19] Ferroni F, Tarleton E and Fitzgerald S 2014 GPU accelerated dislocation dynamics *J. Comput. Phys.* **272** 619–28
- [20] Sills R B and Cai W 2014 Efficient time integration in dislocation dynamics *Modelling Simul. Mater. Sci. Eng.* **22** 025003
- [21] Sills R B, Aghaei A and Cai W 2016 Advanced time integration algorithms for dislocation dynamics simulations of work hardening *Modelling Simul. Mater. Sci. Eng.* **24** 045019

- [22] Cecka C, Lew A J and Darve E 2011 Assembly of finite element methods on graphics processors *Int. J. Numer. Methods Eng.* **85** 640–69
- [23] Sills R B, Bertin N, Aghaei A and Cai W 2018 Dislocation networks and the microstructural origin of strain hardening *Phys. Rev. Lett.* **121** 085501
- [24] Bertin N and Cai W 2018 Energy of periodic discrete dislocation networks *J. Mech. Phys. Solids* **121** 133–46